Documentation de l'outil

Construction de tables d'expérience
Package `tablexp`, méthodes, vérifications et application

Une table d'expérience est l'estimation d'une transition d'un modèle multi-états

Établie le 17 octobre 2026 | Version 0.1 | Statut : Projet | Package `tablexp` 0.0.0.9000

Objet du document

Ce document est la documentation de l'**outil de construction de tables d'expérience** `outil_tables`. Il couvre le package R `tablexp`, qui porte toute la logique quantitative, les vérifications qui l'accompagnent (batteries de tests, contrôles du dépôt, plateforme de calcul) et, quand elle existera, l'application interactive qui l'exploite. Une *table d'expérience* est construite à partir des observations d'un portefeuille d'assurés : pour un risque donné, l'outil estime le taux d'une *transition* (vivant → décédé, en cours → racheté…) selon un ou plusieurs axes, le lisse ou le positionne sur une table de référence, le ferme aux âges extrêmes, le valide, et restitue la table avec tout ce qui permet de la rejouer et de la relire.

Les destinataires des tables et des rapports sont, par priorité, les hypothèses biométriques de la meilleure estimation (*best estimate*) Solvabilité II, que l'ACPR revoit en contrôle, puis la certification d'une table d'expérience par un actuaire indépendant agréé (`docs/exigences.md`, E1.2). Le document doit permettre à un relecteur externe, sans lire le code, de savoir ce qui a été calculé, avec quelles hypothèses, et de rejouer le calcul (E3.7). Les textes qui font foi sont recensés par le cahier des charges (`docs/exigences.md`, § 2.4) ; ils ne sont cités dans le présent document qu'une fois établis sur la source (issue #13).

Il est organisé en une partie par module du package (E3.1). La **partie I** présente l'outil : forme du package et graphe des modules, services transverses, reproductibilité, batteries de vérification et plateforme de calcul, contrôles du dépôt. La **partie II** décrit le point d'entrée unique et le plan de construction. Les parties suivantes suivent l'ordre des modules du moteur : données, exposition, taux bruts, lissage, positionnement, table prospective, fermeture, validation, rapport, puis le simulateur, qui produit les jeux de données sous une loi vraie connue. La dernière partie rassemble l'index des fonctions exportées, la traçabilité des exigences et les références. Chaque partie de module se termine par le tableau des fonctions du module, qui relie la description de la méthode à son implémentation ; un module qui n'existe pas encore a déjà sa partie, avec un tableau vide.

Le document ne porte aucun chiffre issu d'une exécution (E3.4) : les résultats d'une construction sont dans son rapport figé et sa fiche de construction, et les bilans des vérifications dans la sortie des batteries. Ses conventions de rédaction sont fixées par `docs/latex/CONVENTIONS.md` ; le vocabulaire est celui du glossaire du projet (`CONTEXT.md`).

Élément	Référence
Cahier des charges	`docs/exigences.md` (exigences E1.1 à E5.5)
Décisions d'architecture	`docs/adr/` (ADR 0002 à 0008)
Moteur de calcul	package `tablexp` (`R/`), packages de priorité `base` seulement (ADR 0002)
Interface	`app/app.R`, `app/affichage.R` (à venir ; aucun calcul)
Version de R	4.3.3 ; `DESCRIPTION` déclare `Depends: R (>= 4.3.3)`
Plateforme de référence	conteneur `rocker/r-ver:4.3.3` épinglé par empreinte, BLAS et LAPACK de référence (ADR 0008 ; section 4.3)
Données	aucune donnée réelle : jeux simulés sous une loi vraie et une graine

Sommaire

Partie I — Présentation de l'outil 6

1 Architecture du package et graphe des modules 6
1.1 Principe de séparation . . . 6
1.2 Fichiers du package . . . 6
1.3 Graphe d'appels autorisés . . . 7

2 Services transverses : `R/outils.R` 10
2.1 Refus : une condition R typée . . . 10
2.2 Avertissements métier . . . 10
2.3 Fonctions du module . . . 10

3 Reproductibilité et aléa 11
3.1 Exigence et portée . . . 11
3.2 Générateur fixé . . . 11
3.3 Exécution sous graine locale . . . 12
3.4 Vérifications . . . 12

4 Batteries de vérification et plateforme de CI 13
4.1 Trois batteries . . . 13
4.2 Harnais . . . 14
4.3 Plateforme de référence . . . 14
4.4 Jobs de la CI . . . 15

5 Contrôles du dépôt 16
5.1 Scripts de contrôle . . . 16
5.2 Absence de Java . . . 17

Partie II — Point d'entrée et plan de construction 19

6 Point d'entrée unique : `R/construire_table.R` 19
6.1 Plan de construction et validation . . . 19
6.2 Objet résultat . . . 19
6.3 Fonctions du module . . . 19

Partie III — Module `donnees` 20

7 Données, rôles et contrôles : `R/donnees.R` 20
7.1 Conventions de temps . . . 20
7.2 Catalogue des rôles . . . 21
7.3 Contrôles, refus et exclusions . . . 22
7.4 Unité d'observation et fusion par tête . . . 24
7.5 Table de référence . . . 25
7.6 Vérifications . . . 26

7.7 Fonctions du module 26

Partie IV — Module `exposition` 28

8 Exposition et événements : `R/exposition.R` 28
8.1 Recette d'exposition et conventions d'âge 28
8.2 Troncature à gauche et censure à droite 28
8.3 Découpage par année 28
8.4 Fonctions du module 28

Partie V — Module `brut` 29

9 Taux bruts : `R/brut.R` 29
9.1 Fonctions du module 29

Partie VI — Module `lissage` 30

10 Lissage : `R/lissage.R` 30
10.1 Fonctions du module 30

Partie VII — Module `positionnement` 31

11 Positionnement sur une table de référence : `R/positionnement.R` 31
11.1 Fonctions du module 31

Partie VIII — Module `prospective` 32

12 Tables prospectives : `R/prospective.R` 32
12.1 Fonctions du module 32

Partie IX — Module `fermeture` 33

13 Fermeture aux âges extrêmes : `R/fermeture.R` 33
13.1 Fonctions du module 33

Partie X — Module `validation` 34

14 Validation et suivi d'expérience : `R/validation.R` 34
14.1 Fonctions du module 34

Partie XI — Module `rapport` 35

15 Fiche de construction et empreintes : `R/rapport.R` 35
15.1 Fiche de construction 35
15.2 Empreinte des données 35
15.3 Fonctions du module 35

Partie XII — Module `simulation` 36

16 Simulateur : `R/simulation.R` 36
16.1 Loi vraie et tables synthétiques . . . 36
16.2 Portefeuille simulé . . . 36
16.3 Constats de simulation . . . 36
16.4 Fonctions du module . . . 36

Partie XIII — Index et traçabilité 37

17 Index et traçabilité 37
17.1 Index des fonctions exportées . . . 37
17.2 Traçabilité des exigences . . . 37
17.3 Références . . . 38

Partie I

Présentation de l'outil

Architecture du package et graphe des modules

1.1 Principe de séparation

L'outil sépare strictement le *calcul* de l'*affichage*. La règle est la suivante : si un résultat peut être calculé indépendamment de l'interface, son calcul figure dans le package `tablexp` (E4.3). Le moteur reçoit un `data.frame` et la déclaration des *rôles* de ses colonnes, jamais un fichier ; la lecture des fichiers, et les conversions de type qui l'accompagnent, relèvent d'une couche mince hors du moteur, qui n'existe pas encore (E4.4). Le moteur refuse un type incompatible, il ne le convertit pas.

Emplacement	Contenu	Interdit
`R/` (package `tablexp`)	toute la logique quantitative : rôles et validation des données, recettes d'exposition, estimation, lissage, positionnement, prospective, fermeture, validation, simulation, quantités des rapports et empreintes	toute primitive Shiny (`input$`, `reactive`, `render*`) ; tout package hors priorité `base` sans ADR de méthode ; toute dépendance à Java
`app/app.R` (à venir)	déclaration des rôles, édition du plan, appel du point d'entrée sur clic du bouton « Relancer les calculs », affichage	tout calcul ; tout appel d'une fonction non exportée (`tablexp:::`)
`app/affichage.R` (à venir)	formatage, tableaux HTML, tracés à partir des quantités du résultat	tout calcul
`tests/`	batteries, harnais, scripts de contrôle	—

L'application vit hors du package, sous `app/`, et non dans un fichier `R/affichage.R` : tout fichier de `R/` est installé dans l'espace de noms du package (section 1.2), si bien qu'un fichier d'affichage placé là entrerait dans le moteur et rendrait circulaire le contrôle de frontière (ADR 0007, point 1.3). Le dossier `app/` et son script de frontière n'existent pas encore : ils viennent avec la branche de l'application.

1.2 Fichiers du package

Le package `tablexp` est à la racine du dépôt (ADR 0007, point 1.1) :

— `DESCRIPTION` : `Package: tablexp`, `Depends: R (>= 4.3.3)` (décision M44), aucun champ `Imports`, `Suggests` ni `LinkingTo`, `Encoding: UTF-8`, `License: file LICENSE` ; le contrôle de la liste blanche (section 5) en fait une règle vérifiée ;

— `NAMESPACE`, écrit à la main : une directive `export` par fonction, sans `exportPattern` ; les fonctions exportées sont les points d'entrée et les fonctions que l'application appelle (contrôles de saisie à sens métier, constructeurs d'objets partagés), recensées dans l'index (section 17.1) ; aucune méthode S3 n'est exportée ;

— `R/` : un fichier par module et deux fichiers transversaux (tableau 1) ;

— `.Rbuildignore` : exclut de l'archive du package `app/`, `docs/`, `tests/`, `.claude/`, `.github/` et les fichiers de pilotage du dépôt.

Aucun fichier `Rd` ni `roxygen2` : la documentation de référence est le présent document, tenu concordant avec le code par la batterie de concordance (section 4 ; E3.3).

`R/` ne contient que du code du package. Une installation depuis l'arbre de travail (`R CMD INSTALL <dossier>`), qui est celle des batteries, n'applique pas `.Rbuildignore` : tout fichier de `R/` est installé, même s'il y est exclu (constat : un fichier `R/b.R` exclu par `.Rbuildignore` est présent dans le package installé ; ADR 0007, mesures). Un fichier étranger au moteur ne peut donc pas être rangé dans `R/`.

Table 1 : Fichiers de `R/`, dans l'ordre des modules (ADR 0007, points 1.2 et 2.1).

Fichier	Contenu	État
`R/outils.R`	services transverses sans contenu métier : graine locale, refus et avertissements (section 2) ; n'appelle aucun module	présent
`R/donnees.R`	rôles, contrôles, refus et exclusions, unité d'observation, table de référence (section 7)	présent
`R/exposition.R`	recettes d'exposition, conventions d'âge, troncature et censure (section 8)	présent
`R/brut.R`	taux bruts et intervalles (section 9)	à créer
`R/lissage.R`	lissages (section 10)	à créer
`R/positionnement.R`	positionnement sur une table de référence (section 11)	à créer
`R/prospective.R`	tables prospectives (section 12)	à créer
`R/fermeture.R`	fermeture aux âges extrêmes (section 13)	à créer
`R/validation.R`	validation et suivi d'expérience (section 14)	à créer
`R/rapport.R`	quantités des rapports, fiche de construction, empreintes (section 15)	présent
`R/simulation.R`	simulateur (section 16) ; hors de la chaîne du moteur	présent
`R/construire_table.R`	point d'entrée unique, validation du plan, orchestration seule, aucun calcul (section 6)	présent

1.3 Graphe d'appels autorisés

Les modules sont ordonnés : `outils`, `donnees`, `exposition`, `brut`, `lissage`, `positionnement`, `prospective`, `fermeture`, `validation`, `rapport`. Un module ne peut appeler que des fonctions définies dans les modules situés **strictement avant lui** dans cet ordre (ADR 0007, point 2.1). Deux fichiers sont hors de la chaîne : `simulation`, qui ne peut appeler que `donnees` et `outils` et qu'**aucun module du moteur n'appelle**, et `construire_table`, qui peut appeler tous les modules **sauf** `simulation`. La figure 1 et le tableau 2 donnent ce graphe.

Deux raisons fondent cette règle. Un ordre sans cycle rend chaque module lisible et vérifiable avec ses seuls modules amont : une anomalie d'un module ne peut atteindre que les modules situés après lui. Et le simulateur, qui produit les jeux sur lesquels le moteur est testé, ne partage pas le code qu'il sert à tester :

s'il pouvait appeler `exposition`, une erreur de la recette d'exposition se retrouverait à l'identique dans les données simulées et dans leur estimation, et la vérification ne la verrait pas (ADR 0007, « Options écartées »). Le simulateur reproduit le même modèle, pas le même code. Une seule exception à la lecture « en aval seulement » est assumée : `positionnement` peut appeler `lissage`, parce que la méthode 4 de positionnement est un lissage de la table du moment avec dérive de la référence (E2.12) ; le sens inverse est interdit, `lissage` n'appelant jamais `positionnement`.

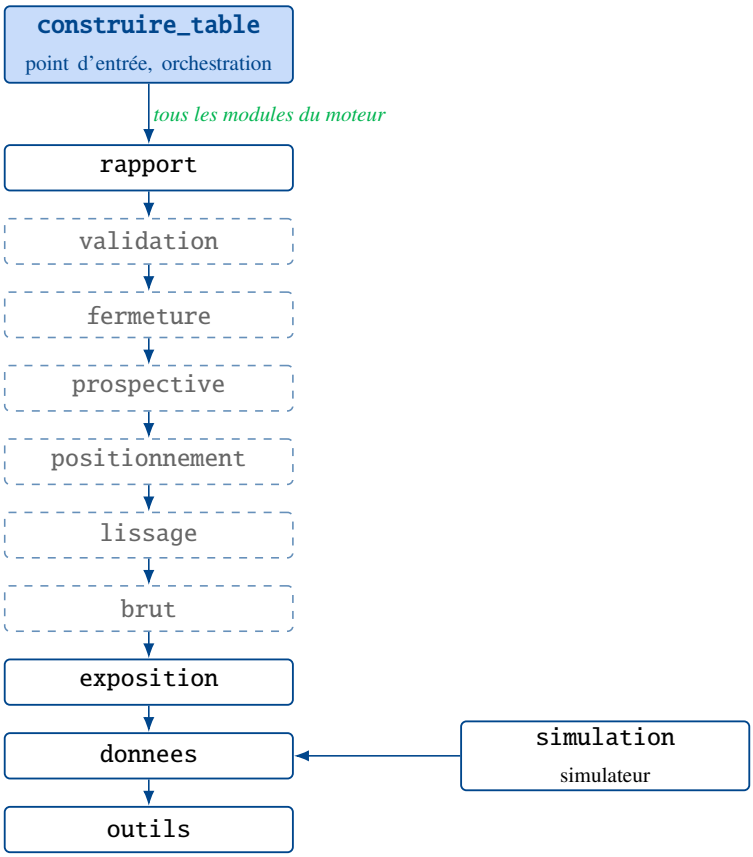


Figure 1 : Graphe d'appels autorisés des modules (ADR 0007, point 2.1). Une flèche de A vers B signifie que A peut appeler B et, la règle étant « strictement avant », tout module situé après B dans le sens des flèches. Blocs pleins : fichiers présents dans `R/` ; blocs en pointillé : modules prévus, fichiers absents. `simulation` n'est appelé par aucun module du moteur, ni par `construire_table` ; `lissage` n'appelle jamais `positionnement`.

Table 2 : Appels autorisés, par module appelant (ADR 0007, point 2.1 ; constante `APPELS_AUTORISES` de `tests/controles/graphe_modules.R`).

Module appelant	Peut appeler
`outils`	aucun module (fonctions des packages de priorité `base` seulement)
`donnees`	`outils`
`exposition`	`donnees`, `outils`
`brut`	`exposition` et les modules amont
`lissage`	`brut` et les modules amont
`positionnement`	`lissage` et les modules amont
`prospective`	`positionnement` et les modules amont
`fermeture`	`prospective` et les modules amont
`validation`	`fermeture` et les modules amont
`rapport`	tous les modules ci-dessus

Module appelant	Peut appeler
`simulation`	`donnees` et `outils` seulement
`construire_table`	tous les modules sauf `simulation`

Contrôle par script

Le graphe est vérifié par `tests/controles/graphe_modules.R`, en R de base (`codetools`, de priorité `recommended`, est exclu par l'ADR 0002), dans le job « Tests (R 4.3.3) » de la CI (ADR 0007, point 2.2) :

1. chaque fichier `R/<module>.R` est lu par `parse()` ; un fichier dont le nom n'est pas un module connu est un écart ;
2. les affectations de premier niveau (`nom <- ...` ou `nom = ...`, fonctions comme constantes) donnent la table « symbole → module » ; un symbole défini dans deux modules est un écart ;
3. pour chaque définition, les noms qu'elle emploie (`all.names()`), croisés avec cette table, donnent les arêtes entre modules ; toute arête absente du tableau 2 est un écart, imprimé avec le module appelant, le module appelé et les deux symboles ;
4. les appels dynamiques interdits (ci-dessous) sont cherchés dans tout l'arbre syntaxique.

Le script imprime toutes les arêtes trouvées et tous les écarts, puis sort avec le code 1 s'il en a trouvé au moins un. Ses limites sont écrites dans son en-tête : un nom local homonyme d'un symbole d'un autre module compte comme une arête (faux positif possible, jamais d'arête manquée par homonymie) ; un code de premier niveau qui n'est pas une affectation est un écart ; les appels vers les packages de base ne sont pas contrôlés ici, mais par la liste blanche de `DESCRIPTION`. Le script est lui-même testé sur des témoins (`tests/unitaires/test_controles.R`) : il échoue, entre autres, quand `exposition` appelle `brut`, quand `donnees` appelle `simulation`, quand `lissage` appelle `positionnement` et quand `construire_table` appelle `simulation`, et il passe sur un graphe conforme où `positionnement` appelle `lissage`.

Appels dynamiques interdits

Un appel dont la cible est calculée à l'exécution (`do.call("f", ...)`, `get("f")`) n'apparaît pas dans `all.names()` : il créerait une arête invisible pour le contrôle. Le script interdit donc dans `R/` (ADR 0007, point 2.3, liste élargie à la mise en œuvre et consignée dans l'annotation du 16/10/2026) :

— `do.call()` dont l'argument `what`, apparié par `match.call()` (position ou nom), n'est ni un symbole ni une fonction anonyme (chaîne, `paste0(...)...`) ; `do.call(f, args)` avec `f` un symbole désignant une fonction reste permis ;

— toute occurrence, appel ou fonction passée comme objet (`lapply(x, get)`), de `match.fun`, `get`, `get0`, `mget`, `getFromNamespace`, `getExportedValue`, `getFunction`, `asNamespace` et `getNamespace` ;

— `eval()` ou `evalq()` d'un `parse()`, `str2lang()`, `str2expression()`, `call()` ou `as.call()` ;

— l'opérateur `:::` et toute référence qualifiée `tablexp::` ;

— les formes qualifiées `base::`, `utils::` et `methods::` de ce qui précède, lues comme la forme nue.

Une fonction passée en argument puis appelée dans le corps de la fonction qui la reçoit crée une arête que le script attribue à l'appelant qui l'a passée : c'est voulu, la dépendance existe bien là. Les tests, seuls, appellent les fonctions non exportées par `tablexp:::` (ADR 0007, point 11.3).

Services transverses : R/outils.R

R/outils.R porte les services sans contenu métier partagés par tous les modules : l'exécution sous graine locale (section 3), la forme des refus et celle des avertissements (ADR 0007, point 1.2). Il n'appelle aucun module. Sans lui, la fonction de graine locale serait dupliquée dans le simulateur et dans le futur bootstrap par têtes, et chaque module inventerait sa forme de refus.

2.1 Refus : une condition R typée

Le moteur ne devine pas : quand une entrée ne permet pas de construire, il *refuse* (E2.3, E2.7). Un refus est une **condition R typée, jamais un objet retourné** (ADR 0007, point 6.4). refus_tablexp() construit la condition, de classe c("refus_tablexp", "error", "condition"), avec quatre champs :

— module : le module qui refuse ;

— motif : un code court stable, en minuscules ASCII, chiffres et soulignés, conforme au motif MOTIF_CODE_REFUS (^[a-z][a-z0-9_]*$) ;

— message : la phrase qui dit ce qui manque (E5.5) ;

— details : une liste (rôle, colonne, étape, paramètre, décompte, premiers numéros de ligne).

refuser() la lève par stop() et ne rend jamais la main. Un argument mal formé de refus_tablexp() (module ou message vide, motif non conforme, détails qui ne sont pas une liste) n'est pas un refus mais une erreur de programmation du package, signalée par stop(). Les tests vérifient le motif, le décompte et les numéros par tryCatch() ; l'application attrapera la condition et affichera son message. Une condition est préférée à un objet portant ok = FALSE, forme d'outil_USP : un tel objet peut être lu par mégarde comme une construction partielle, une condition ne le peut pas. Les codes de refus de chaque module sont recensés dans la partie du module.

2.2 Avertissements métier

Un avertissement métier signale ce qui n'empêche pas la construction mais doit être lu : unité d'observation par défaut, exclusion appliquée, relevé d'incohérence non bloquant. Il est **inscrit dans l'objet résultat, jamais émis par warning()** (ADR 0007, point 6.5) : l'objet est la seule source du rapport figé et de l'application ; un warning() serait perdu par l'application, absent de la fiche et parasite dans les batteries. avertissements_vides() crée l'accumulateur, un data.frame de colonnes etape, code, message et nombre ; avertir() lui ajoute une ligne, et refuse, par une erreur de programmation, un code qui n'est pas conforme à MOTIF_CODE_REFUS. Chaque fonction d'étape rend ses avertissements ; le point d'entrée les rassemble dans le champ avertissements de l'objet et de la fiche. Les codes de chaque étape sont recensés dans la partie de son module (sections 7.4 et 8).

2.3 Fonctions du module

Table 3 : Définitions de premier niveau de `R/outils.R`.

Nom	Statut	Entrées	Sortie et rôle
`GENERATEUR`	constante	—	générateur fixé (section 3.2) : Mersenne-Twister, inversion pour la loi normale, rejet pour `sample()`
`MOTIF_CODE_REFUS`	constante	—	motif d'un code de refus ou d'avertissement : `^[a-z][a-z0-9_]*$`
`refus_tablexp()`	interne	`module`, `motif`, `message`, `details` (liste, vide par défaut)	condition de classe `refus_tablexp`, sans la lever ; `stop()` si un argument est mal formé (section 2.1)
`refuser()`	interne	mêmes arguments	lève le refus ; ne rend jamais la main
`avertissements_vides()`	interne	—	accumulateur vide des avertissements métier (section 2.2 ; issue #9)
`avertir()`	interne	accumulateur, `etape`, `code`, `message`, `nombre`	accumulateur augmenté d'une ligne, sans `warning()` (section 2.2 ; issue #9)
`sous_graine()`	interne	`graine` (entier explicite, sans défaut), `expr`	valeur de `expr` évaluée sous la graine et `GENERATEUR`, état du générateur de l'appelant restauré ; refus `graine_invalide` (section 3.3)

Reproductibilité et aléa

3.1 Exigence et portée

À données, rôles, plan et graine identiques, le programme produit des résultats identiques **au bit près sur une même machine** ; toute simulation passe par une graine locale explicite sous un générateur fixé, consignée dans le résultat, l'état global du générateur étant restauré à la sortie ; aucun horodatage n'entre dans la partie du résultat comparée lors d'un rejeu (E4.6). La présente section décrit les deux mécanismes du socle : le générateur fixé et l'exécution sous graine locale. La consignation de la graine et du générateur dans la fiche de construction et le rejeu de la fiche sont décrits avec la fiche (section 15.1).

> **Statut.** L'identité au bit près est un *constat* des batteries (section 3.4), établi sur la machine qui les exécute ; ce n'est pas une propriété démontrée. L'identité entre deux machines n'est **pas garantie** : deux processeurs différents pourraient différer au dernier bit, même dans le même conteneur, si la bibliothèque mathématique du système choisit ses variantes de calcul selon le processeur (hypothèse de l'ADR 0008, non mesurée). C'est pourquoi les références de non-régression seront comparées à tolérance et produites sur une plateforme unique (section 4.3).

3.2 Générateur fixé

Le générateur est la constante `GENERATEUR` de `R/outils.R` : `kind = "Mersenne-Twister"`, `normal.kind = "Inversion"`, `sample.kind = "Rejection"`. Ce sont les valeurs par défaut de R 4.3.3 (`RNGkind()` dans une session neuve), fixées en constante pour qu'un changement de défaut d'une version de R ne change pas les tirages en silence. Le générateur uniforme est le Mersenne-Twister de Matsumoto et Nishimura ; les lois normales s'obtiennent par inversion de la fonction de répartition ; la génération uniforme discrète (`sample()`) emploie la méthode par rejet, défaut de R depuis la version

3.6.0, la méthode antérieure par arrondi rendant `sample()` sensiblement non uniforme sur de grandes populations (documentation de R, `?Random`).

Référence. M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`).

Un changement de `GENERATEUR` change tout résultat simulé : c'est un changement de résultat, soumis au visa (ADR 0007, « Conséquences »).

3.3 Exécution sous graine locale

`sous_graine(graine, expr)` est la seule voie par laquelle le package tire un aléa (ADR 0007, point 8.1). Elle procède ainsi :

1. **contrôle de la graine** : un nombre unique, non manquant, fini, entier et de valeur absolue au plus `.Machine$integer.max` ; sinon, et aussi quand la graine est omise, refus `graine_invalide` du module `outils`. Il n'y a jamais de graine par défaut : une graine par défaut serait une graine non déclarée ;
2. **sauvegarde de l'état de l'appelant** : `.Random.seed` de l'environnement global, ou son absence, et le réglage `RNGkind()` ;
3. **tirage** : `set.seed(graine, kind, normal.kind, sample.kind)` sous `GENERATEUR`, puis évaluation de `expr` ;
4. **restauration**, par `on.exit()`, donc aussi après une erreur dans `expr` : le réglage d'abord, parce que `RNGkind()` réécrit `.Random.seed`, puis `.Random.seed` de l'appelant, ou sa suppression s'il n'existait pas.

Les tirages ne dépendent donc que de la graine : ni de l'état du générateur de l'appelant, ni de son réglage, que `sous_graine()` laisse inchangés. Le simulateur (section 16) est, à cette date, la seule fonction du package qui tire un aléa ; il reçoit sa graine en argument et la consigne, avec `GENERATEUR`, dans le jeu simulé.

Limite. La réserve de la méthode normale de Box-Muller d'un appelant, gardée par R hors de `.Random.seed`, n'est pas restaurable ; c'est sans effet sur le moteur, qui tire toujours sous `Inversion` (limite reprise d'`outil_USP`, ADR 0007, point 8.1).

Aucun aléa hors calcul. Les fichiers temporaires, employés par exemple pour calculer une empreinte par `tools::md5sum()`, sont créés par `tempfile()`, qui ne touche pas `.Random.seed` (constat : dans une session neuve, `.Random.seed` n'existe ni avant ni après `tempfile()` et `tools::md5sum()` ; ADR 0007, point 8.2).

3.4 Vérifications

Deux fichiers de tests portent ces propriétés (section 4) :

— `tests/unitaires/test_outils.R` : valeur de `GENERATEUR` ; mêmes tirages que `set.seed()` nu sous le générateur par défaut ; graine entière écrite en double admise ; appelant sous `L'Ecuyer-CMRG` : tirages de `GENERATEUR`, `.Random.seed` et `RNGkind()` de l'appelant restaurés ; absence de `.Random.seed` restaurée ; état restauré après une erreur dans `expr` ; refus `graine_invalide` des graines mal formées ;

— `tests/test_reproductibilite.R` : à graine égale, tirages identiques (`identical()`) dans un même processus, quel que soit l'état du générateur de l'appelant, et dans un autre processus R de la même machine ; à graines différentes, tirages différents. Les tirages comparés couvrent les trois réglages du générateur (`runif()`, `rnorm()`, `sample.int()`) ; le processus fils les écrit en `%.17g`, format qui restitue un double au bit près. La même batterie vérifie le simulateur et la construction (sections 16 et 15).

Ces vérifications sont des constats sur la machine qui exécute la batterie ; sur la CI, c'est la plateforme de référence (section 4.3).

Batteries de vérification et plateforme de CI

4.1 Trois batteries

Les vérifications du code passent par trois batteries en R de base, sans `testthat` (package du CRAN, hors priorité `base`), lancées depuis la racine du dépôt (ADR 0007, point 11.1) :

```
Rscript tests/test_unitaires.R
Rscript tests/test_reproductibilite.R
Rscript tests/concordance_doc_moteur.R --strict
```

La même liste, dans le même ordre, figure dans `CLAUDE.md` (« Commandes »), dans le workflow `.claude/workflows/circuit-technique.js` et dans le job « Tests (R 4.3.3) » de la CI ; un script le contrôle (section 5). Chaque batterie :

— **installe l'arbre de travail** dans une bibliothèque temporaire (`R CMD INSTALL --no-docs -l <bibliothèque> <racine>`, par `installer_tablexp()`), sans installer aucun autre package, puis charge `tablexp` depuis elle et échoue s'il est chargé d'ailleurs : c'est le code de l'arbre qui est testé, jamais un package installé auparavant ;

— affiche `R.version.string` puis, à la fin, une **ligne de bilan** : version de R, nombre de tests, d'échecs, d'échecs attendus et de succès inattendus ;

— **sort avec un code non nul** en cas d'échec ou de succès inattendu (section 4.2).

Une bibliothèque déjà installée, désignée par la variable d'environnement `TABLEXP_BIBLIOTHEQUE`, n'est réutilisée que si l'*empreinte de l'arbre source* inscrite à son installation est égale à celle de l'arbre courant ; sinon la batterie réinstalle et le dit. Cette empreinte est un MD5 (`tools::md5sum()`) de `DESCRIPTION`, de `NAMESPACE` et, s'ils existent, de tous les fichiers de `R/`, `inst/`, `data/` et `src/`, chemins relatifs compris, triés par octets (`empreinte_source()`). Elle empêche de tester un package périmé ; elle n'a aucun rôle dans les résultats.

`tests/test_unitaires.R` exécute chaque fichier `tests/unitaires/test_*.R`, triés par nom, chacun dans son propre environnement ; `quit()`, `q()` et `bilan()` y sont neutralisés, une erreur au chargement d'un fichier est un échec, et un fichier sans aucune assertion aussi. **`tests/test_reproductibilite.R`** vérifie les identités de la section 3.4, puis celles du simulateur et de la construction ; la comparaison aux références de non-régression s'y ajoutera avec la branche qui les crée. **`tests/concordance_doc_moteur.R`** confronte le présent document au code installé : toute fonction citée existe ; toute fonction exportée est citée, ou inscrite dans la liste d'attente `tests/concordance_attente.csv` avec renvoi à l'issue qui la documentera, où elle compte en échec attendu ; toute ligne de cette liste désigne un export existant et renvoie à une issue. Le format des citations et des catalogues qu'elle lit est fixé par `docs/latex/CONVENTIONS.md` (section 3). Sans `--strict`, elle rend compte des écarts sans échouer.

4.2 Harnais

Les trois batteries partagent `tests/harnais.R`, repris de la forme d'`outil_USP` (ADR 0007, point 11.2, et annotation du 16/10/2026, point d) :

— `verifier(nom, expr)` : succès si `expr` vaut `TRUE` ; une chaîne décrit l'écart ; toute autre valeur, ou une erreur, est un échec ;

— `echec_attendu(nom, issue, expr, motif, erreur_quelconque)` : **défaut connu**, codé en échec attendu avec renvoi obligatoire à l'issue, de la forme `#N` ; `expr` décrit le comportement *correct*. Tant qu'elle échoue, le test est « attendu » ; **si elle passe, c'est un succès inattendu et la batterie échoue** : le défaut semble corrigé, la marque est retirée et le test devient ordinaire. Trois natures d'échec attendu sont admises et décomptées : par valeur (`expr` rend `FALSE` ou une chaîne), par refus (un `refus_tablexp`, du motif donné s'il l'est) et, seulement si `erreur_quelconque = TRUE`, par une autre erreur ; toute autre issue est un échec, pour qu'un défaut connu ne masque pas une panne différente ;

— `proche(a, b, rel, abs)` : égalité numérique à tolérance déclarée, $|a - b| \leq abs + rel\,|b|$ élément par élément, sans tolérance par défaut ; rend un écart pour des valeurs non numériques, des longueurs différentes ou des valeurs manquantes à des positions différentes ; exige l'égalité exacte des valeurs infinies ;

— `leve_refus(expr, motif)` : vrai si `expr` lève un `refus_tablexp` du motif donné ;

— `sans_execution(x)` : retire de l'objet résultat le champ des informations d'exécution, seul champ qui change entre deux exécutions identiques, avant `identical()` ;

— `bilan()` : ligne de bilan et code de sortie. Il compte en outre un échec si un espace de noms hors priorité `base`, autre que `tablexp`, est chargé en fin de batterie (`espaces_hors_base()`) : l'image de la CI contient des packages `recommended`, et cette règle empêche un test de s'appuyer sur l'un d'eux à l'insu de la liste blanche (ADR 0008, « Conséquences »).

Une **garde de sortie** (`.Last()`) fait sortir avec le code 1 tout processus qui a chargé le harnais et se termine avant la ligne de bilan, même par `quit(status = 0)`. Le harnais est lui-même testé (`tests/unitaires/test_harnais.R`) : une assertion fausse donne un code de sortie non nul, et un échec attendu qui réussit fait échouer la batterie.

Chaque test nomme la nature de son attendu (ADR 0007, point 11.4 ; E2.20) : valeur calculée à la main (identité exacte, ou tolérance déclarée) ; *recoupement*, comparaison à la valeur que rend, dans la batterie, un appel direct d'une fonction d'un package de priorité `base`, dont la portée est limitée quand le moteur repose sur la même fonction : le test dit alors ce qu'il vérifie (la formule et son branchement, non la fonction de base) ; *contre-vérification*, valeur de référence produite hors CI par un package avancé et écrite en dur avec sa version et sa date ; *constat de simulation*, qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (`CONTEXT.md` ; E2.19).

4.3 Plateforme de référence

Les références de non-régression, qui viendront avec la deuxième branche, sont produites et comparées sur une *plateforme de référence* unique (E4.7 ; ADR 0008) : le job de la CI s'exécute dans un **conteneur `rocker/r-ver:4.3.3` épinglé par empreinte**, dont R charge la **BLAS et la LAPACK de référence** (Netlib), en monothread. Identifiant fixé par l'annotation du 16/10/2026 de l'ADR 0008 (décision M47) :

> image `rocker/r-ver:4.3.3@sha256:`
> `732d15020af326da9e919c07f70ca32bf5d3e409220af32e0a4b6d0a89437309`
> suivie de la bascule `update-alternatives` vers les paquets `libblas3` et `liblapack3` 3.10.0-2ubuntu1 d'Ubuntu 22.04 ;

```
R version 4.3.3 (2024-02-29);
BLAS /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0;
LAPACK /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0, version 3.10.0;
monothread (OPENBLAS_NUM_THREADS et OMP_NUM_THREADS valent 1).
```

Pourquoi un conteneur épinglé. Un libellé d'exécuteur (`ubuntu-24.04`) n'est pas une image figée : elle est mise à jour par GitHub, bibliothèques comprises, sans commit dans le dépôt. `outil_USP`, qui installait R sur l'image de l'exécuteur, a constaté une dérive entre deux exécutions de même R et de même image, due à une BLAS optimisée (OpenBLAS) qui choisit ses noyaux selon le processeur (constat d'`outil_USP` rapporté par l'ADR 0008, « Contexte »). Une étiquette de conteneur peut elle aussi désigner une autre image après une reconstruction (hypothèse de l'ADR 0008, non vérifiée) ; l'empreinte (`@sha256:...`) désigne un contenu immuable. Le conteneur est une fonctionnalité native de GitHub Actions, et non une action tierce, ce que la politique des Actions du dépôt exige.

Pourquoi la bascule. L'image charge par défaut OpenBLAS (relevé de la première exécution, rapporté par l'annotation du 16/10/2026 de l'ADR 0008). Le mainteneur a retenu de basculer dans le job vers la Netlib déjà présente dans l'image, plutôt que de compiler R depuis la source (décision M47 ; le repli par compilation reste prévu par l'ADR 0008, point 10). L'étape « BLAS et LAPACK de référence » de `.github/workflows/ci.yml` pointe par `update-alternatives` les alternatives `libblas.so.3-x86_64-linux-gnu` et `liblapack.so.3-x86_64-linux-gnu` vers `libblas3` et `liblapack3`, puis affiche les cibles résolues et les versions des paquets. La plateforme de référence est l'image épinglée *suivie de cette bascule* : modifier l'étape, l'image ou les bibliothèques est un changement de plateforme, décidé par le mainteneur et, une fois des références existantes, accompagné d'un tableau avant / après de toutes les références (ADR 0008, point 9).

Script de plateforme. `tests/controles/plateforme.R` s'exécute en première étape de chaque job R, avant tout calcul (ADR 0008, points 3 à 5). En mode strict (`--exiger`, sur la CI), il **échoue** si : `R.version.string` n'est pas exactement `R_ATTENDU` (`R version 4.3.3 (2024-02-29)`) ; l'une des quatre lectures de la BLAS et de la LAPACK (`extSoftVersion()["BLAS"]`, `sessionInfo()$BLAS`, `sessionInfo()$LAPACK`, `La_library()`) est vide, désigne une BLAS optimisée (motif `MOTIF_OPTIMISE` : `openblas`, `mkl`, `atlas`, `blis`, `flexiblas`) ou sort des chemins de référence (`MOTIF_BLAS`, `MOTIF_LAPACK`) ; `La_version()` n'est pas `LA_VERSION_ATTENDUE` (3.10.0) ; `OPENBLAS_NUM_THREADS` ou `OMP_NUM_THREADS` ne vaut pas 1 ; `TABLEXP_IMAGE`, que le job renseigne avec la même chaîne que l'image du conteneur, n'est pas épinglée par empreinte (`MOTIF_IMAGE`). Sans `--exiger` (poste, session), il affiche les mêmes grandeurs et les écarts sans échouer : les batteries restent exécutables sur le poste Windows du mainteneur, dont la BLAS est celle de R. Dans les deux modes, il écrit l'*identifiant de plateforme* (image, `R.version.string`, BLAS, LAPACK et sa version, `sessionInfo()`) et consigne sans faire échouer le processeur et ses drapeaux (`fma`, `avx2`, `avx512f`), `ImageOS` et `ImageVersion` de l'hôte, `LC_COLLATE`, le fuseau horaire, la présence d'un exécutable `java` et `GITHUB_SHA`. Ce bloc sera reporté dans le message de chaque commit de référence.

> **Ce que la plateforme ne garantit pas.** Le modèle de processeur varie d'une exécution à l'autre sous le même libellé d'exécuteur (relevé de l'ADR 0008, annotation du 16/10/2026). La BLAS Netlib n'a pas de noyau choisi selon le processeur ; la réserve sur la bibliothèque mathématique du système (section 3.1) reste en revanche ouverte, non mesurée. Les références seront donc comparées à tolérance, structure contrôlée à part (E4.7).

4.4 Jobs de la CI

`.github/workflows/ci.yml` définit les jobs suivants, exécutés sur chaque pull request, sur chaque push vers `main` et sur demande. Aucun job n'installe de package du CRAN ni ne configure de dépôt

CRAN pour une installation : `tablexp` est installé depuis l'arbre de travail par chaque batterie.

Table 4 : Jobs de la CI (`.github/workflows/ci.yml`).

Job	Exécution	Contenu
« Contrôles du dépôt »	hôte `ubuntu-24.04`, sans R	réglages de Claude Code valides (JSON) ; fiches d'agents complètes ; fiches `-approfondi` à jour ; aucun fichier sensible versionné
« Tests (R 4.3.3) »	conteneur de la plateforme de référence	bascule vers la BLAS et la LAPACK de référence ; script de plateforme en mode strict ; les trois batteries ; liste blanche de `DESCRIPTION` ; ordre des modules ; identité des batteries et de l'image. Les batteries et les contrôles s'exécutent même si une étape antérieure échoue (condition `${{ !cancelled() }}`), pour que tous les écarts soient visibles ; le job reste en échec
« Absence de Java »	même conteneur, réseau requis	contrôle de l'absence de Java (section 5.2) et son cas témoin

Ces trois jobs sont les contrôles requis que liste le ruleset de `main` (`.github/ruleset-main.json`, appliqué au dépôt par le mainteneur) : une pull request ne peut alors être fusionnée que s'ils sont verts. L'ADR 0008 (point 6) prévoit un quatrième job, « Documentation », qui compilera le PDF du présent document sans action tierce et le publiera en artefact ; il n'existe pas encore. Les batteries et les scripts des jobs conteneurisés ne dépendent pas de git, absent de l'image ; un contrôle qui lit l'historique reste dans « Contrôles du dépôt », sur l'hôte (ADR 0008, point 11).

Contrôles du dépôt

5.1 Scripts de contrôle

Tout contrôle mécanique est un script, jamais une relecture (`CLAUDE.md`, « Workflows », principe 7). Les scripts sont en R de base, sous `tests/controles/`, et sortent avec un code non nul en cas d'écart. Chacun est testé sur des témoins dans `tests/unitaires/test_controles.R` : il doit échouer sur un cas fautif et passer sur le dépôt.

Table 5 : Scripts de contrôle (`tests/controles/`).

Script	Règle contrôlée	Source
`liste_blanche_description.R`	`Depends` : exactement `R (>= 4.3.3)`, plus d'éventuels packages de priorité `base` ou admis ; `Imports` : packages de priorité `base` de R 4.3.3, ou admis ; `LinkingTo`, `Suggests`, `Enhances` : packages admis seulement. Un package est admis s'il figure, avec son champ et l'ADR de méthode qui l'introduit, dans la constante `ADMIS`, vide à cette date	ADR 0002, point 3 et A8 ; M44

Script	Règle contrôlée	Source
`graphe_modules.R`	graphe d'appels autorisés et appels dynamiques interdits (section 1.3)	ADR 0007, points 2.1 à 2.3
`identite_batteries.R`	mêmes batteries, dans le même ordre, dans `CLAUDE.md`, dans `circuit-technique.js` et dans les étapes « Batterie : » du job « Tests (R 4.3.3) » ; ce job ne porte ni `continue-on-error`, ni condition `if:` de job, et une étape de batterie n'a pas d'autre condition que `${{ !cancelled() }}` ; une seule chaîne d'image dans tous les workflows	`CLAUDE.md` ; ADR 0008, point 1
`plateforme.R`	plateforme de référence (section 4.3)	ADR 0008, points 3 à 5
`absence_java.R`	aucune dépendance à Java (section 5.2)	ADR 0002, A1 ; ADR 0008, point 7 ; issue #3

La liste des packages de priorité `base` est écrite en dur dans `liste_blanche_description.R` pour R 4.3.3 (`PACKAGES_BASE_R433`) : le verdict ne dépend pas de la machine ; le script signale, sans échouer, un écart avec les packages de priorité `base` effectivement installés.

5.2 Absence de Java

Règle. Aucun code du projet ne dépend de Java, ni directement ni indirectement : ni le moteur, ni l'application, ni la contre-vérification (ADR 0002, annotation du 16/10/2026, A1, décision M40). Java n'est pas utilisable sur le poste de travail du mainteneur ; c'est pour cette raison que le package `ELT`, qui dépend de `xlsx` et donc de Java, est exclu de la contre-vérification. Sa source ne peut être exécutée qu'hors du dépôt, sans Java, en appoint, sans qu'aucun code en soit copié (ADR 0002, A5 et troisième annotation).

Contrôle. `tests/controles/absence_java.R` (issue #3) contrôle trois ensembles de packages :

— le *moteur* : les packages déclarés dans les champs `Depends`, `Imports` et `LinkingTo` de `DESCRIPTION`, hors R ; aucun à cette date ;

— l'*application* : `shiny`, `htmltools`, et `openxlsx` à titre facultatif ;

— la *contre-vérification* : les packages `survival`, `mgcv`, `WH`, `StMoMo`, `demography`, `MortalityLaws`, `lifecontingencies`, `survPen`, `MortCast`, `locfit`, et `flexsurv` à titre facultatif.

Les deux derniers ensembles sont lus dans une liste unique, le fichier `packages_hors_moteur.csv` du dossier `tests/controles/` (colonnes `ensemble`, `package`, `statut`, `source`), qui est la référence ; la liste ci-dessus en est la transcription à la date du document. Le script lit les métadonnées d'un **instantané daté** du CRAN chez Posit Package Manager, par `available.packages()`, sans rien installer ; la date est la constante `INSTANTANE` du script (2026-10-15, un jour ouvré : selon la documentation de Posit citée par l'ADR 0008, aucun instantané n'est publié les week-ends et jours fériés), qui ne change que par un commit `repo:`. Il calcule, pour chaque package, la fermeture récursive de ses dépendances `Depends`, `Imports` et `LinkingTo`, par la fonction `tools::package_dependencies()` appelée avec `recursive = TRUE`, et **échoue** (code 1) :

— si un package de `PACKAGES_JAVA` (`rJava`, `xlsx`, `xlsxjars`) figure dans un ensemble ou dans la fermeture de l'un de ses packages ; la chaîne de dépendances la plus courte qui y mène est affichée ;

— si un package listé, ou une dépendance récursive hors priorité `base`, manque aux métadonnées : un package introuvable n'a pas de dépendances connues, il ne passe pas en silence ;

— si la liste est mal formée (en-tête, ensembles et statuts connus, noms valides, sans doublon, ensembles non vides).

Des métadonnées illisibles (panne réseau, adresse fausse) donnent le code 2 : aucun verdict n'est rendu, l'échec du job n'est jamais masqué et une relance le lève. Seul le filtre `duplicates` d'`available.packages()` est appliqué : le filtre par version de R écarterait les packages qui exigent un R plus récent que celui qui exécute le contrôle, alors que la contre-vérification peut tourner sur un autre poste muni d'un R récent, et ces packages seraient alors signalés absents (faux échec) ; le filtre par système d'exploitation est écarté aussi, le poste étant sous Windows et la CI sous Linux.

Cas témoin. Le job « Absence de Java » exécute aussi le script avec `--ajouter ELT` : il doit rendre le code 1 (un verdict, et non le code 2 d'une panne) et afficher la chaîne de `ELT` à `xlsx`. Les tests du script s'exécutent sans réseau, sur des dépôts locaux témoins (`tests/unitaires/test_controles.R`).

Limite. La détection porte sur les seuls noms de `PACKAGES_JAVA`. Le champ `SystemRequirements`, qui pourrait déclarer Java, n'est pas renseigné dans les métadonnées `PACKAGES` du CRAN (relevé consigné dans l'en-tête du script) : un package qui exigerait Java sans dépendre de l'un de ces noms échapperait au contrôle. Étendre la détection relève du mainteneur.

PARTIE II

Point d'entrée et plan de construction

Point d'entrée unique : R/construire_table.R

6.1 Plan de construction et validation

À rédiger (issue #11).

6.2 Objet résultat

À rédiger (issue #11).

6.3 Fonctions du module

À rédiger (issue #11).

PARTIE III

Module donnees

Données, rôles et contrôles : R/donnees.R

Le module donnees reçoit la base d'observation, un data.frame, et la déclaration des rôles de ses colonnes. Il contrôle les rôles, évalue les motifs d'invalidité de chaque ligne, applique les règles d'exclusion déclarées par le plan, déduit l'unité d'observation et fusionne les intervalles d'observation d'une même tête (E2.2, E2.4, E2.7). Il ne modifie jamais les données : une ligne invalide arrête la construction, sauf règle d'exclusion déclarée, et rien n'est corrigé (E2.7). Sa fonction d'étape est preparer_donnees() ; il porte aussi le catalogue des rôles (catalogue_roles()) et la représentation commune d'une table de référence (table_reference()), toutes deux exportées. Le contenu de fond (catalogue, motifs, règles de fusion) vient de la spécification d'actuary du 16/10/2026, versée dans l'issue #11 (parties A.0 et A.1).

7.1 Conventions de temps

Trois conventions de temps sont communes à toutes les recettes (E2.3) ; elles sont appliquées par donnees et par exposition (section 8), qui ajoute l'unité de l'exposition et la fenêtre d'observation.

— **Une date est un jour** (classe Date, sans heure ni fuseau). Un intervalle d'observation est l'intervalle d'instants semi-ouvert $[\text{entrée}, \text{sortie})$: date_entree est l'instant où l'observation commence, date_sortie celui où elle cesse (jour du décès ou du rachat, ou lendemain du dernier jour couvert pour une fin de garantie). Un événement daté du jour s survient à l'instant s : la tête est exposée jusqu'à s, jour s exclu. Une ligne en_cours est ouverte à droite. *Choix de conception* : il donne des décomptes entiers et additifs, au prix d'une sous-estimation de l'exposition d'un demi-jour par événement en moyenne.

— **Âge au dernier anniversaire.** L'âge au jour t d'une tête née le jour n est le nombre d'années révolues

$$\text{age}(n,t) = \text{an}(t) - \text{an}(n) - \mathbb{1}\{(\text{mois}(t), \text{jour}(t)) < (\text{mois}(n), \text{jour}(n))\},$$

comparaison lexicographique, appliquée sans cas particulier (age_revolu()). Pour une naissance le 29 février, l'anniversaire tombe donc le 1er mars des années non bissextiles : le 28 février d'une telle année, $(2, 28) < (2, 29)$ et l'année n'est pas révolue. Une année d'âge dure ainsi 365 ou 366 jours. Convention de calcul, conséquence de la formule (spécification, A.0, T2), validée par actuary le 17/10/2026, d'impact négligeable (un jour d'exposition déplacé entre deux âges, pour les seules naissances du 29 février) ; aucune source juridique n'est établie à son sujet.

— **Période d'observation et délai de déclaration.** La période $[\text{début}, \text{fin}]$ est déclarée par le plan, fin incluse, jamais déduite des données (E2.5). Le délai de déclaration est un nombre entier de jours ≥ 0,

sans arithmétique de mois. La fin effective est fin − délai et la borne d'instant B = fin effective + 1 : l'observation s'arrête à l'instant B. Une période dont le début est postérieur à la fin effective est refusée (`periode_vide`).

7.2 Catalogue des rôles

Le moteur ne lit une colonne que par son *rôle*, déclaré par l'utilisateur dans un catalogue fermé (E2.2 ; ADR 0003, point 2). Les rôles sont transmis à `construire_table()` sous la forme d'un vecteur de chaînes nommé : noms = rôles du catalogue, valeurs = noms de colonnes de la base (ADR 0007, point 4.1). Les colonnes sans rôle sont ignorées. `catalogue_roles()`, exportée pour que l'application propose la déclaration des rôles, rend le catalogue sous la forme d'un `data.frame` de colonnes `role`, `famille`, `type_admis`, `usage` et `recettes` (tableau 6).

La colonne *usage* décrit l'emploi du rôle par la seule recette d'exposition disponible, `lignes_dates_completes` (section 8.1) :

— `requis` : rôle sans lequel aucune recette ne s'applique (refus `aucune_recette`) ;

— `facultatif` : rôle lu pour des contrôles (unicité du contrat, cohérence de l'année de naissance) ou, pour `id_tete`, pour l'unité d'observation ;

— `segmentation` : rôle requis si la segmentation du plan l'emploie (refus `role_requis_manquant`), contrôlé s'il est seulement déclaré, avec l'avertissement `role_non_employe` ;

— `non_pris_en_charge` : rôle qu'aucune recette disponible n'emploie et dont l'omission changerait le sens du résultat (pondération, agrégats, états, date de survenance distincte de la sortie, date d'inventaire, âge fourni) ; sa déclaration est **refusée** (`role_non_pris_en_charge`) plutôt qu'ignorée en silence. Le rôle `montant` est de ceux-là : la pondération par les montants attend la spécification de ses intervalles (E2.8).

Les types sont contrôlés sans conversion (E4.4) : une colonne de dates doit être de classe `Date`, exactement, à valeurs entières et finies ; une colonne `POSIXct`, une chaîne ou un nombre sont refusés (`type_incompatible`), la conversion revenant à la couche de lecture. Les modalités des rôles catégoriels sont comparées après conversion en chaîne, en respectant la casse : `H` et `F` pour `sexe` ; `deces`, `rachat`, `terme`, `autre` et `en_cours` pour `cause_sortie` (constantes `MODALITES_SEXE` et `MODALITES_CAUSE`). Un recodage (par exemple `1/2` vers `H/F`) relève de la couche de lecture.

Table 6 : Catalogue fermé des rôles (`catalogue_roles()` ; constantes `CATALOGUE_ROLES` et `TYPES_ADMIS`). La colonne `recettes` du catalogue vaut `lignes_dates_completes` pour tout rôle pris en charge, vide sinon.

Rôle	Famille	Types R admis	Usage
`id_contrat`	identification	`character`, `integer`, `double` entier fini	facultatif
`id_tete`	identification	`character`, `integer`, `double` entier fini	facultatif
`sexe`	état civil	`character`, `factor`	segmentation
`date_naissance`	état civil	`Date` (valeurs entières finies)	requis
`annee_naissance`	état civil	`integer`, `double` entier fini	facultatif
`age_fourni`	état civil	`integer`, `double` entier fini ≥ 0	non pris en charge
`date_age_fourni`	état civil	`Date` (valeurs entières finies)	non pris en charge
`date_entree`	observation	`Date` (valeurs entières finies)	requis
`date_sortie`	observation	`Date` (valeurs entières finies ; manquante admise pour une ligne `en_cours`)	requis

Rôle	Famille	Types R admis	Usage
cause_sortie	observation	character, factor	requis
date_evenement	observation	Date (valeurs entières finies)	non pris en charge
date_inventaire	observation	Date (valeurs entières finies)	non pris en charge
age	agrégats	integer, double entier fini	non pris en charge
annee	agrégats	integer, double entier fini	non pris en charge
exposition	agrégats	double fini ≥ 0	non pris en charge
evenements	agrégats	double fini ≥ 0	non pris en charge
montant	pondération	double fini > 0	non pris en charge
segment	segmentation	character, factor, integer	segmentation
etat_depart	multi-états	character, factor	non pris en charge
etat_arrivee	multi-états	character, factor	non pris en charge
date_changement	multi-états	Date (valeurs entières finies)	non pris en charge

7.3 Contrôles, refus et exclusions

Refus au niveau des rôles et des paramètres

controler_roles() contrôle la déclaration des rôles dans cet ordre, et chaque refus nomme tous les rôles et colonnes en cause : donnees_mal_formees (l'entrée n'est pas un data.frame) ; roles_mal_formes (les rôles ne sont pas un vecteur de chaînes nommé, sans valeur ni nom manquant) ; role_inconnu ; role_declare_deux_fois ; role_non_pris_en_charge ; colonne_absente ; colonne_ambigue (le nom de colonne figure plusieurs fois dans la base : lire la première sans le dire serait deviner) ; colonne_declaree_deux_fois (une colonne pour deux rôles) ; type_incompatible ; aucune_recette (un rôle requis manque ; le message nomme la recette et les rôles absents) ; role_requis_manquant (un rôle employé par la segmentation n'est pas déclaré).

controler_parametres_donnees() contrôle les champs du plan que l'étape reçoit : transition_inconnue (liste de champs autres qu'exactement depart et arrivee, ou transition non prise en charge : seules vivant $\rightarrow$ décédé et en cours $\rightarrow$ racheté le sont, constante TRANSITIONS) ; periode_invalide (liste de champs autres qu'exactement debut et fin, ou dates manquantes ou non Date) ; delai_invalide (délai qui n'est pas un entier ≥ 0) ; periode_vide ; segmentation_invalide (vecteur de chaînes sans doublon pris dans sexe et segment, vide explicite admis) ; exclusions_invalides ; exclusion_inconnue (code absent de la liste fermée des motifs). Les noms de champs sont lus exactement, jamais par appariement partiel. Le même contrôle est appelé par valider_plan(), avant toute étape : un code désigne une condition, quelle que soit la fonction qui la détecte, et le refus porte le module donnees (ADR 0007, annotation du 17/10/2026, point d).

Motifs d'invalidité des lignes

Les motifs forment une liste fermée (E2.7), évaluée par evaluer_motifs() (tableau 7).

— **Exhaustivité** : tous les motifs de ligne sont évalués sur toutes les lignes, sans court-circuit ; un motif de comparaison n'est pas levé si l'une de ses dates est manquante (le motif de la valeur manquante l'est) ;

— **motifs de tête** : évalués, quand id_tete est déclaré, sur les lignes exemptes de motif de ligne, ils s'appliquent à toutes les lignes de la tête ; le rapport est ainsi complet en un seul passage ;

— **décompte** : une ligne peut porter plusieurs motifs ; le décompte par motif compte les lignes porteuses, et le nombre de lignes distinctes est donné à part.

Table 7 : Motifs d'invalidité, dans l'ordre de restitution (constantes `MOTIFS_LIGNES` et `MOTIFS_TETE`). Ce sont aussi les codes admis comme règles d'exclusion.

Code	Condition
Motifs de ligne	
`id_contrat_manquant`	`id_contrat` déclaré et manquant
`id_tete_manquant`	`id_tete` déclaré et manquant
`naissance_manquante`	date de naissance manquante
`entree_manquante`	date d'entrée manquante
`sortie_manquante`	date de sortie manquante et cause autre que `en_cours`
`cause_manquante`	cause de sortie manquante
`cause_hors_modalites`	cause hors des cinq modalités
`sexe_manquant`	`sexe` déclaré et manquant
`sexe_hors_modalites`	`sexe` déclaré, hors de `H` et `F`
`segment_manquant`	`segment` déclaré et manquant
`naissance_apres_entree`	naissance postérieure à l'entrée (l'égalité est admise)
`sortie_avant_entree`	sortie antérieure à l'entrée (l'égalité est admise : durée nulle)
`en_cours_avant_fin`	cause `en_cours` et date de sortie renseignée antérieure à la fin déclarée : la base ne couvre pas la période déclarée
`annee_naissance_incoherente`	`annee_naissance` déclarée et différente de l'année de la date de naissance
`age_aberrant`	âge au dernier anniversaire au moins égal à `AGE_ABERRANT` (130) à la plus petite des dates de sortie et de fin de période (à la fin si la sortie manque) ; constante de catalogue, *choix de conception* : la borne laisse passer toute durée de vie humaine connue, elle n'est pas un paramètre du plan
`contrat_duplique`	`id_contrat` déclaré et présent sur plusieurs lignes (toutes les lignes du groupe sont listées)
Motifs de tête (si `id_tete` est déclaré)	
`tete_naissance_incoherente`	dates de naissance différentes entre lignes d'une même tête
`tete_sexe_incoherent`	sexes différents au sein d'une tête (`sexe` déclaré)
`tete_segment_incoherent`	segments différents au sein d'une tête, si la segmentation emploie `segment`
`tete_observee_apres_deces`	le premier décès s_d de la tête est antérieur à B et une ligne de la tête a une fin d'observation postérieure à s_d (lecture B de E2.4, ci-dessous)

Refus par défaut et exclusions déclarées

Une ligne qui porte un motif **non déclaré** dans les règles d'exclusion du plan arrête la construction (refus `ligne_invalide`), même si elle porte aussi un motif déclaré. Le message nomme, pour chaque motif en cause, le nombre de lignes et leurs dix premiers numéros (`NB_LIGNES_CITEES`, numéros 1 à n des lignes du `data.frame` reçu), puis le total des lignes distinctes ; la condition porte la liste complète (E2.7, E5.5).

Le plan peut déclarer des *règles d'exclusion*, paramètre `exclusions` de l'étape `donnees` : un vecteur de codes de motif sans doublon, vide explicite admis. Une ligne est exclue si l'un de ses motifs est déclaré. Les exclusions sont comptées et listées par motif dans le résultat de l'étape (`resultat$exclusions`, `resultat$lignes_exclues`) et chacune donne un avertissement `exclusion`. Si toutes les lignes sont exclues, la construction est refusée (`aucune_ligne_conservee`) ; une base sans ligne l'est aussi (`base_vide`). Le diagnostic `diagnostics$motifs` restitue, pour chaque motif, sa nature (ligne ou tête), son décompte et s'il est exclu.

Les lignes conservées sont celles de l'entrée : `donnees` lui-même s'il n'y a aucune exclusion, sinon le sous-ensemble de ses lignes, sans aucune modification de valeur ; les quantités dérivées (observations

fusionnées) sont rangées à part. Une réserve technique : l'extraction des lignes conservées par l'opérateur d'indexation de R retire des colonnes les attributs autres que la classe et les niveaux d'un facteur (par exemple `tzone`, `label` ; constat sur R 4.3.3) ; les valeurs et les classes ne changent pas, et le moteur ne lit pas ces attributs.

7.4 Unité d'observation et fusion par tête

Unité d'observation, déduite des rôles

L'unité d'observation n'est pas un paramètre du plan : le moteur la déduit des rôles et de la transition, comme il déduit la recette (décision M48 ; ADR 0007, annotation du 17/10/2026, point a), et l'inscrit dans les paramètres effectifs de l'étape et dans la fiche :

— **tête**, si l'état de départ de la transition est un état de la personne (vivant → décédé) et que `id_tete` est déclaré ;

— **contrat** sinon : transition dont l'état de départ est un état du contrat (en cours → racheté, décision M46), ou transition de tête sans `id_tete`. Dans ce second cas, l'avertissement `unite_contrat` est inscrit dans l'objet et repris dans la fiche : « une tête titulaire de plusieurs contrats est comptée plusieurs fois, ce qui surestime l'exposition et peut compter plusieurs fois un même décès » (cas F4 des tests : deux décès comptés au lieu d'un).

Un paramètre d'unité serait soit redondant avec les rôles, soit en contradiction avec eux : une tête déclarée « contrat » alors que `id_tete` est présent compterait plusieurs fois un même décès. Pour le rachat, l'unité est le contrat ; l'identifiant de tête sert alors aux contrôles de cohérence et au futur rééchantillonnage par têtes (E2.4, E2.14) : il est gardé dans la table des observations.

Fusion des intervalles d'une tête

En unité tête, les intervalles $[\text{entrée}, \text{fin})$ des lignes d'une même tête (fin $= +\infty$ pour une ligne `en_cours`) sont triés par entrée et fusionnés quand entrée suivante $\leq$ fin courante, ce qui couvre le chevauchement et la contiguïté exacte (`fusionner_intervalles()`). **Aucune tolérance « à un jour près »** : un écart d'un jour reste un trou non exposé, et la tête est de nouveau tronquée à gauche à sa réentrée ; combler l'écart inventerait une exposition. Un tel écart trahit souvent une date de fin inclusive dans la base : il est relevé et décompté à part, non corrigé (*choix de conception*). La cause de fin d'une composante fusionnée est `deces` si une ligne de décès se termine à cette fin, `en_cours` si la fin est $+\infty$, `censure` sinon ; le décès n'est compté qu'une fois.

Le calcul de la fusion est vectorisé par un maximum cumulé décalé par tête, exact en `double` tant que (nombre de têtes) $\times$ (étendue des dates, en jours, $+1$) reste inférieur à 2^{52}. Au-delà, le moteur refuse (`dates_hors_plage`) au lieu de fusionner inexactement ; pour 10^6 têtes, il faudrait une étendue d'environ $4{,}5 \cdot 10^9$ jours, ce que seules des dates aberrantes atteignent. C'est une limite d'implémentation, non un motif de ligne : elle n'est pas excluable.

Observation après le décès : lecture B de E2.4

Une tête dont un contrat est observé après son décès a une trajectoire indéterminée, et ses lignes sont refusées (E2.4). Le décès doit cependant être connu : le mainteneur a retenu une *lecture B* (décision M48) : le motif `tete_observee_apres_deces` n'est levé que si le premier décès s_d de la tête est **antérieur à la borne** B de la période effective et qu'une fin d'observation d'une ligne de la tête le dépasse. Une ligne `en_cours` compte pour $+\infty$, quelle que soit sa date de sortie, et une sortie manquante aussi (`deces_par_tete()`). Deux lignes closes par le décès le même jour sont cohérentes.

Si $s_d \geq B$, le décès est hors de la période effective : aucune ligne n'est refusée, mais le relevé observations_apres_deces_hors_periode compte ces têtes et donne un avertissement du même code. Ce relevé est calculé **sur les seules lignes conservées** : une tête écartée par une exclusion déclarée n'y est pas comptée, une tête dont une ligne seulement est exclue l'est sur ses lignes conservées. Il est calculé dès que id_tete est déclaré, **quelle que soit l'unité**, donc aussi en unité contrat pour le rachat (contrôle par identifiant de tête, E2.4, M46, M48) ; sans id_tete, il n'est pas calculable et vaut NA.

Relevés et avertissements

L'étape restitue dans diagnostics$releves, sans bloquer, le nombre de têtes et de têtes multi-contrats, de chevauchements, de contiguïtés, d'intervalles disjoints, d'écarts d'un jour, de fenêtres vides (observations sans intersection avec la période), d'observations, et le relevé après décès hors période. Les relevés de fusion valent NA sans id_tete. Ses avertissements sont : unite_contrat ; role_non_employe ; exclusion ; chevauchements, disjoints et ecarts_un_jour, en unité tête seulement ; fenetres_vides et observations_apres_deces_hors_periode, dans toutes les unités. Les contiguïtés sont décomptées sans avertissement : une contiguïté exacte est fusionnée sans perte.

Le résultat de l'étape (resultat) contient les lignes conservées et leurs numéros, les exclusions, l'unité retenue, la table des observations (une ligne par composante fusionnée ou par contrat : identifiant, numéro de la première ligne, nombre de lignes, date de naissance, sexe s'il est déclaré, segment s'il est employé, début, fin, cause) et, pour chaque ligne conservée, le numéro de son observation. Les paramètres effectifs (parametres) sont le code interne de la transition (vivant_decede ou en_cours_rachete), l'unité d'observation, la segmentation, les exclusions et la fin effective.

7.5 Table de référence

Toute table de référence — loi vraie synthétique du simulateur, table embarquée, table importée — est lue par le moteur sous une seule forme : la classe S3 table_reference, sur un data.frame en format long (ADR 0007, point 7.1, et annotation du 17/10/2026, point b), que construit et contrôle table_reference(x, nom, statut, source, parametres, forme_origine), exportée pour l'import de tables par l'application :

— colonnes age (entier ≥ 0), sexe (H, F, ou ensemble pour une table sans distinction de sexe, modalité qui ne coexiste jamais avec H ou F, dans la table ou au sein d'une même année), qx (double fini dans $[0, 1]$) et, facultative, annee (entier), pour les tables prospectives de la v2 ;

— âges sans doublon, croissants dans l'ordre reçu et contigus, au sein de chaque sexe (et de chaque année) ;

— attributs nom, statut (synthetique, embarquee ou importee), source (arrêté, URL, fichier de provenance), forme_origine (qx ou lx, défaut qx) et parametres : une liste non vide est **requise** pour une table synthétique, qui n'a d'autre provenance que sa loi, et **interdite** pour une table embarquée ou importée, qu'elle ferait passer pour dérivée d'une loi.

Le constructeur refuse au lieu de corriger, avec un code par contrôle : table_nom_manquant, table_statut_inconnu, table_source_manquante, table_parametres_manquants, table_parametres_inattendus, table_forme_inconnue, table_mal_formee, table_colonne_manquante, table_colonne_inconnue, table_vide, table_colonne_mal_typee, table_sexe_hors_modalites, table_sexes_melanges, table_qx_non_fini, table_qx_hors_bornes, table_age_double, table_ages_non_croissants et table_ages_non_contigus.

7.6 Vérifications

tests/unitaires/test_donnees.R vérifie le catalogue, chaque refus de rôle et de paramètre, chaque motif et les règles d'évaluation, les exclusions (lignes conservées identiques à l'entrée), la lecture B de E2.4 au jour près de la borne B (délais 0 et 30 jours) et le relevé après décès hors période (lignes conservées, deux unités), la table de référence et ses refus. Les cas de fusion F1 à F7 de la spécification (chevauchement, contiguïté, écart d'un jour, décès sur deux contrats, observation après décès, deux décès, naissances discordantes) ont des attendus calculés à la main ; la fusion est vérifiée en outre contre un second algorithme, une boucle naïve écrite dans le test.

7.7 Fonctions du module

Table 8 : Définitions de premier niveau de R/donnees.R.

Nom	Statut	Entrées	Sortie et rôle
CATALOGUE_ROLES	constante	—	catalogue fermé des rôles (rôle, famille, code de type, usage)
TYPES_ADMIS	constante	—	libellé des types R admis par code de type
RECETTE_BRANCHE	constante	—	seule recette d'exposition disponible : lignes_dates_completes
MODALITES_SEXE	constante	—	H, F
MODALITES_CAUSE	constante	—	deces, rachat, terme, autre, en_cours
ROLES_SEGMENTATION	constante	—	sexe, segment
AGE_ABERRANT	constante	—	130 (motif age_aberrant)
TRANSITIONS	constante	—	transitions prises en charge : départ, arrivée, code interne, porteur de l'état de départ (tête ou contrat)
MOTIFS_LIGNES	constante	—	motifs de ligne (tableau 7)
MOTIFS_TETE	constante	—	motifs de tête (tableau 7)
NB_LIGNES_CITEES	constante	—	10 numéros de ligne cités par motif dans un message de refus
catalogue_roles()	exportée	—	catalogue des rôles (tableau 6)
composantes_date()	interne	jours depuis 1970-01-01	année, mois, jour
age_revolu()	interne	naissance, date	âge au dernier anniversaire (section 7.1)
entiers_finis()	interne	vecteur	valeurs non manquantes entières et finies
type_conforme()	interne	colonne, code de type	conformité au type du catalogue, sans conversion
liste_guillemets()	interne	chaînes	liste entre guillemets pour les messages
message_aucune_recette()	interne	rôles requis, manquants	message du refus aucune_recette, partagé avec exposition
controler_roles()	interne	données, rôles, segmentation	refus au niveau des rôles ; rend les rôles inchangés
controler_parametres_donnees()	interne	transition, période, délai, segmentation, exclusions	refus des paramètres ; transition retenue, début, fin, borne B
groupes_heterogenes()	interne	groupes, valeurs	groupes où la valeur n'est pas constante
extremes_par_groupe()	interne	groupes, valeurs	minimum et maximum par groupe
deces_par_tete()	interne	têtes, sorties, causes	premier décès de chaque tête et observation au-delà

Nom	Statut	Entrées	Sortie et rôle
evaluer_motifs()	interne	données, rôles, période, borne, segmentation	numéros des lignes porteuses de chaque motif
fusionner_intervalles()	interne	têtes, entrées, fins	composantes fusionnées et relevés ; refus dates_hors_plage
preparer_donnees()	interne	données, rôles, transition, période, délai, segmentation, exclusions	fonction d'étape donnees : paramètres effectifs, résultat, diagnostics, avertissements
STATUTS_TABLE	constante	—	synthetique, embarquee, importee
SEXES_TABLE	constante	—	H, F, ensemble
FORMES_ORIGINE	constante	—	qx, lx
table_reference()	exportée	table, nom, statut, source, paramètres, forme d'origine	objet table_reference contrôlé (section 7.5)

PARTIE IV

Module exposition

Exposition et événements : R/exposition.R

8.1 Recette d'exposition et conventions d'âge

À rédiger (issue #11).

8.2 Troncature à gauche et censure à droite

À rédiger (issue #11).

8.3 Découpage par année

À rédiger (issue #11).

8.4 Fonctions du module

À rédiger (issue #11).

PARTIE V

Module brut

Taux bruts : R/brut.R

Module prévu (E2.12, E2.14) : estimateurs des taux bruts et leurs intervalles de confiance exacts et asymptotiques. Il sera créé par la branche des taux bruts (docs/feuille-de-route.md, § 2).

9.1 Fonctions du module

TABLE 9 : Définitions de premier niveau de R/brut.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/brut.R absent			

PARTIE VI

Module lissage

Lissage : R/lissage.R

Module prévu (E2.12) : lissages des taux bruts. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

10.1 Fonctions du module

TABLE 10 : Définitions de premier niveau de R/lissage.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/lissage.R absent			

PARTIE VII

Module positionnement

Positionnement sur une table de référence : R/positionnement.R

Module prévu (E2.12, E2.15) : positionnement d'une table d'expérience sur une table de référence. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

11.1 Fonctions du module

TABLE 11 : Définitions de premier niveau de R/positionnement.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/positionnement.R absent			

Partie VIII

Module prospective

Tables prospectives : R/prospective.R

Module prévu (E2.13, v2) : tables prospectives, toute table prospective étant une surface de mortalité.

12.1 Fonctions du module

Table 12 : Définitions de premier niveau de R/prospective.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/prospective.R absent			

PARTIE IX

Module fermeture

Fermeture aux âges extrêmes : R/fermeture.R

Module prévu (E2.12) : fermeture de la table aux âges extrêmes. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

13.1 Fonctions du module

TABLE 13 : Définitions de premier niveau de R/fermeture.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/fermeture.R absent			

PARTIE X

Module validation

Validation et suivi d'expérience : R/validation.R

Module prévu (E2.12, E2.18) : validation de la table construite et suivi d'expérience. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

14.1 Fonctions du module

TABLE 14 : Définitions de premier niveau de R/validation.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/validation.R absent			

PARTIE XI

Module rapport

Fiche de construction et empreintes : R/rapport.R

15.1 Fiche de construction

À rédiger (issue #11).

15.2 Empreinte des données

À rédiger (issue #11).

15.3 Fonctions du module

À rédiger (issue #11).

PARTIE XII

Module simulation

Simulateur : R/simulation.R

16.1 Loi vraie et tables synthétiques

À rédiger (issue #10).

16.2 Portefeuille simulé

À rédiger (issue #10).

16.3 Constats de simulation

À rédiger (issue #10).

16.4 Fonctions du module

À rédiger (issue #10).

Partie XIII

Index et traçabilité

Index et traçabilité

17.1 Index des fonctions exportées

Le tableau 15 recense les fonctions exportées de `tablexp` (directives `export` de `NAMESPACE` ; ADR 0007, point 3.4) : les points d'entrée et les fonctions que l'application appelle. Les autres fonctions sont internes ; elles figurent dans le tableau des fonctions de leur module.

Table 15 : Fonctions exportées de `tablexp`.

Fonction	Module	Rôle	Section
`catalogue_roles()`	`donnees`	catalogue fermé des rôles, lu par l'application pour la déclaration des rôles	7.2
`table_reference()`	`donnees`	constructeur contrôlé d'une table de référence, appelé par l'application pour l'import de tables	7.5

17.2 Traçabilité des exigences

Le tableau 16 relie les exigences du cahier des charges (`docs/exigences.md`) aux sections qui les décrivent. Il est complété à chaque section rédigée.

Table 16 : Exigences et sections qui les décrivent.

Exigence	Objet	Section
E1.3	rigueur, traçabilité, reproductibilité	3, 4
E2.2	catalogue fermé des rôles, lecture d'une colonne par son rôle	7.2
E2.4	unité d'observation, fusion par tête, incohérences entre contrats	7.4
E2.5	période déclarée, délai de déclaration	7.1
E2.7	refus par défaut, motifs en liste fermée, exclusions déclarées, données inchangées	7.3
E3.1	document LaTeX unique, une partie par module, charte d'`outil_USP`, tableaux de fonctions	ensemble du document ; 2.3
E3.3	test de concordance entre la documentation et le code	4.1
E3.4	aucun chiffre issu d'une exécution	Objet du document
E4.1	R 4.3.3, package `tablexp`, modules	1.2
E4.2	packages de priorité `base` ; aucune dépendance à Java	5.1, 5.2
E4.3	frontière calcul / affichage	1.1

Exigence	Objet	Section
E4.4	le moteur reçoit un `data.frame` et des rôles	1.1
E4.6	reproductibilité, graine locale, générateur fixé	3
E4.7	références de non-régression sur la plateforme de référence	4.3
E4.9	un module par famille, index des fonctions contrôlé par la concordance	1, 4.1
E5.5	messages qui disent ce qui manque, sans rien corriger	2.1, 7.3

17.3 Références

— M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`) ; section 3.2.

— R Core Team, *R : A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienne, 2024 (`https://www.R-project.org/` ; référence rendue par `citation()` sous R 4.3.3).